

VisualizingLQCD.jl 改良提案 v7

現行コード構造、全関数 I/O、依存関係、失敗条件、移行契約を詳細化する

作成: OpenAI GPT-5.5 Pro

2026 年 5 月 4 日

要旨

VisualizingLQCD.jl は、ILDG 形式などの格子 QCD ゲージ配位を読み込み、プラケット由来の局所量を 3 次元等値面として動画化する軽量な可視化パッケージである。本提案 v7 は、前版の方針をさらに実装寄りに整理する。第一に、動画画面から yoctosecond と第 4 方向スライスの表示を外し、Euclidean 配位の切片列を実時間発展に見せない。第二に、プラケットの対数を削らず、等値面の閾値を安定に選ぶための「閾値座標」として正式に採用する。第三に、I/O、flow、observable、transform、level selection、renderer、metadata、配位生成を分離し、各部品を単体テスト可能にする。結論として、最初の改修は小さな PR に分け、互換 API を残しつつ、表示の意味、再現性、保守性を同時に改善する。

目次

1	対象と設計ゴール	4
2	現状コードの整理	4
3	現行コード構造と全関数の I/O	5
3.1	ファイル単位の構造	5
3.2	現行の実行時データフロー	6
3.3	全関数の入出力型一覧	7
3.4	局所関数と匿名関数	8
3.5	top-level 定数と実行時副作用	9
3.6	I/O 型から見える改修方針	9
4	現行実装の詳細棚卸し	9
4.1	呼び出しグラフ	9
4.2	外部 API 依存の書き出し	10

4.3	<code>create_animation</code> の内部変数と形状	11
4.4	<code>heatbath_SU3!</code> と <code>heatbathtest_4D</code> の内部変数	12
4.5	関数ごとの失敗条件と <code>validation</code>	13
4.6	現行処理を段階名へ写した疑似コード	14
4.7	推奨する戻り値契約	15
4.8	推奨する関数シグネチャー一覧	16
4.9	<code>metadata</code> に現行値から書き出せる項目	16
4.10	最小修正で追加できる実装片	18
5	現行コードのマジックナンバーを参考値として残す	18
5.1	基本方針	18
5.2	可視化経路の参考値	18
5.3	描画と画面構成の参考値	19
5.4	スケール設定と配位生成の参考値	20
5.5	README と <code>test</code> の参考値	20
6	時間表示を止める	21
6.1	物理的な理由	21
6.2	表示と <code>metadata</code> の分担	22
7	対数変換を閾値座標として設計する	22
7.1	raw 量と表示変換を分ける	22
7.2	符号の向きを仕様に入れる	23
7.3	ϵ を隠さない	23
7.4	level の逆変換を必ず残す	24
8	閾値選択を安定化する	24
8.1	分位点を既定にする	24
8.2	global level を既定にする	25
8.3	閾値感度を診断する	26
9	部品独立性を高めるアーキテクチャ	26
9.1	責務分離の原則	26
9.2	推奨ファイル構造	26
9.3	中核データ型	27
10	高水準 API と互換性	28
10.1	新しい API	28
10.2	既存 <code>create_animation</code> の扱い	29

11	物理量の拡張	29
11.1	単一面プラケットを互換 observable として残す	29
11.2	全 6 面プラケット平均を既定候補にする	29
11.3	Clover energy density と topological charge density	30
11.4	Gradient flow を表示 observable の一部として記録する	30
12	metadata schema	30
12.1	JSON sidecar の最小要件	30
12.2	タイトルと metadata の対応	31
13	renderer と backend の独立性	31
13.1	Renderer は配列と座標だけを見る	31
13.2	GLMakie と CairoMakie を分ける	32
14	I/O と配位生成の分離	32
14.1	I/O は配位を返すだけにする	32
14.2	配位生成は examples または submodule へ移す	32
15	テスト計画	33
15.1	軽量単体テスト	33
15.2	重い統合テストを分ける	33
16	段階的 PR 計画	33
16.1	PR 0: 表示の誤解をなくす	33
16.2	PR 1: 対数変換と level selection を部品化する	34
16.3	PR 2: metadata sidecar を追加する	34
16.4	PR 3: pipeline を分割する	34
16.5	PR 4: observable を増やす	34
17	リスクと対策	35
18	コードスケルトンの内容	35
19	結論	36
付録 A	README 追記案	37
付録 B	受け入れ基準チェックリスト	37

1 対象と設計ゴール

本メモは、2026 年 5 月 4 日に確認した GitHub main branch の公開コードを対象にする。README は、ILDG 形式の配位を Julia で可視化し、時間方向を実時間方向のように扱い、プラケット、つまり場の強さの等値面を描くと説明している [1]。一方、通常の格子 QCD 配位は Euclidean 経路積分のサンプルであり、動画のフレーム送りは Minkowski 実時間発展ではない。このため、画面から時間表示を外し、説明も「4 次元 Euclidean 配位の 3 次元スライス列」へ変える。

現状の `visualization.jl` は、ILDG ファイルを読み、 xy 面の 1×1 プラケットを評価し、各点で

$$p_{12}(x) = 1 - \frac{1}{N_c} \text{Re tr } U_{12}(x) \quad (1)$$

を作る。その後、 $-\log(p_{12} + 10^{-7})$ を 4 次元配列へ入れ、その変換後の値で等値面レベルを決めている [4]。したがって、対数変換は実装上すでに重要な役割を持っている。本提案では、これを偶然的処理ではなく、等値面の閾値を安定化するための表示変換として設計に組み込む。

v7 の主要決定

- 動画画面には yoctosecond、 t 、 τ 、slice index を既定では表示しない。
- raw plaquette deviation p と display transform $f(p)$ を別のデータとして扱う。
- $-\log(p + \epsilon)$ は削除せず、閾値を置くための座標として残す。
- 変換の向きにより、上側レベルが raw の大きい側か小さい側かを明示する。
- 閾値は変換後の分位点を既定にし、raw equivalent value を metadata に保存する。
- renderer は Gaugefields の更新や ILDG I/O を知らず、observable は Makie を知らない構造にする。
- 配位生成は可視化の必須経路から外し、例またはサブ module として扱う。

2 現状コードの整理

トップレベル module は、Gaugefields、Wilsonloop、Makie、GLMakie、Statistics、StatsBase などを一括で読み込み、`configuration_generation.jl` と `visualization.jl` を include している [3]。Project.toml は Julia 1.10 互換で、Gaugefields.jl、Wilsonloop.jl、Makie/GLMakie、Plots、StatsBaseなどを主要依存としている [2]。この構造では、可視化だけを使う利用者にも配位生成と GL backend の重い依存が見えやすい。

可視化側の `create_animation` は、`ILDG(filename)` と `load_gaugefield!` でゲージ配位を読み、 $[(1,+1),(2,+1),(1,-1),(2,-1)]$ という xy 面の Wilson line を評価する [4]。その後、 p_{12} を計算し、 $-\log(p_{12} + 10^{-7})$ を作り、平均と標準偏差から等値面レベルを選ぶ。最後に GLMakie の `contour!` と `record` で mp4 を生成し、現状では各フレームのタイトルに yoctosecond 換算を

表示する [4]。

配位生成側の `heatbathtest_4D` は、cold start からゲージ場を初期化し、平均ブラケットと Polyakov loop を表示し、20 回のヒートバス更新を行い、その後 `Gradientflow(U)` と `flow!(U,g)` でゲージ場を滑らかにして保存する [5]。このデモは便利だが、可視化 pipeline の中核ではない。`Gaugefields.jl` 自体は $SU(N_c)$ ゲージ場、ゲージ作用、MPI、自動微分を扱い、v0.5.0 以降で NVIDIA GPU サポートも掲げる格子 QCD 基盤 package である [10]。

3 現行コード構造と全関数の I/O

3.1 ファイル単位の構造

現行の `src` directory には、`VisualizingLQCD.jl`、`configuration_generation.jl`、`constants.jl`、`header.jl`、`visualization.jl` が置かれている [7]。ただし、package module が実際に include するのは `configuration_generation.jl` と `visualization.jl` の 2 ファイルだけである。`header.jl` と `constants.jl` は repository 上には残っているが、現行の `VisualizingLQCD.jl` からは include されていない。`configuration_generation.jl` と `visualization.jl` の冒頭にも `include("header.jl")` と `include("constants.jl")` がコメントアウトされた形で残っているため、この 2 ファイルは旧 standalone 実行時代の補助ファイルと見るのが自然である [4, 5]。

`VisualizingLQCD.jl` は module 宣言、依存 package の `using`、2 つの `include`、そして `end` だけで構成される。ここで `Gaugefields`、`Wilsonloop`、`Makie`、`GLMakie`、`Plots`、`StatsBase` などが一括で読み込まれるため、可視化だけを使う場合でも配位生成、描画 backend、統計処理が同じ namespace に入る [3]。現行の public API は `export` 文から見ると `create_animation` と `heatbathtest_4D` の 2 つであり、`automatic_level2`、`ln_a`、`calculate_a`、`heatbath_SU3!` は module 内の非 `export` 関数である [4, 5]。

表 1: 現行 repository のファイル構造と責務。

ファイル	現行の役割	注意点
<code>Project.toml</code>	package 名、uuid、version、依存 package、compat を定義する。	Julia compat は 1.10 で、 <code>GLMakie</code> 、 <code>Gaugefields</code> 、 <code>Wilsonloop</code> などが通常依存に入る [2]。
<code>src/VisualizingLQCD.jl</code>	module 本体で、依存 package を読み、2 つの実装ファイルを include する。	<code>header.jl</code> と <code>constants.jl</code> は include しない。

ファイル	現行の役割	注意点
src/visualization.jl	level 診断、lattice spacing 変換、ILDG 読み込み、plaquette 計算、対数変換、Makie 動画生成を持つ。	GLMakie.activate!() が top-level で実行される。create_animation が export される。
src/configuration_generator.jl	SU(3) heatbath 更新、cold start からの 4 次元純ゲージ配位生成、gradient flow、binary save を持つ。	heatbathtest_4D が export される。heatbath_SU3! は非 export である。
src/header.jl	using 群を列挙する旧補助ファイル。	現行 module からは読まれない。内容は module 本体の using とほぼ重複する [8]。
src/constants.jl	demo 用の $24^3 \times 32$ 、 $\beta = 6.0$ 、 $N_c = 3$ 、file 名を定義する旧補助ファイル。	現行 module からは読まれない。top-level 変数であり const ではない [9]。
test/runtests.jl	test() を定義し、配位生成と動画生成をまとめて実行する。	軽量な数値単体テストではなく、生成と rendering を含む統合 smoke test である [6]。

3.2 現行の実行時データフロー

現行の可視化経路は、create_animation という 1 つの関数の中で完結する。入力 は格子サイズ (N_X, N_Y, N_Z, N_T) 、 N_c 、動画 file 名、 β 、ILDG file 名である。関数は Initialize_Gaugefields で入れ物を作り、ILDG(filename) と load_gaugefield! で file を読み、Wilsonline([(1,+1),(2,+1),(1,-1),(2,-1)]) を評価する [4]。その後、各格子点で $p_{12} = 1 - \Re \text{tr} U_{12}/N_c$ を計算し、 $-\log(p_{12} + 10^{-7})$ を plaqs_t へ格納し、automatic_level2 の統計量から level 列を作り、GLMakie で動画を書き出す。

現行の配位生成経路は、heatbathtest_4D が high-level driver になっている。入力は格子サイズ、 β 、 N_c 、flow step 数、保存 file 名である。関数は cold start のゲージ場を作り、heatbath_SU3! を 20 回呼び、5 sweep ごとに plaquette と Polyakov loop を表示し、最後に gradient flow を flow_steps_in 回かけて save_binarydata で保存する [5]。このため、confname は出力 file であり、戻り値の plaq_t は最後に測った plaquette である。

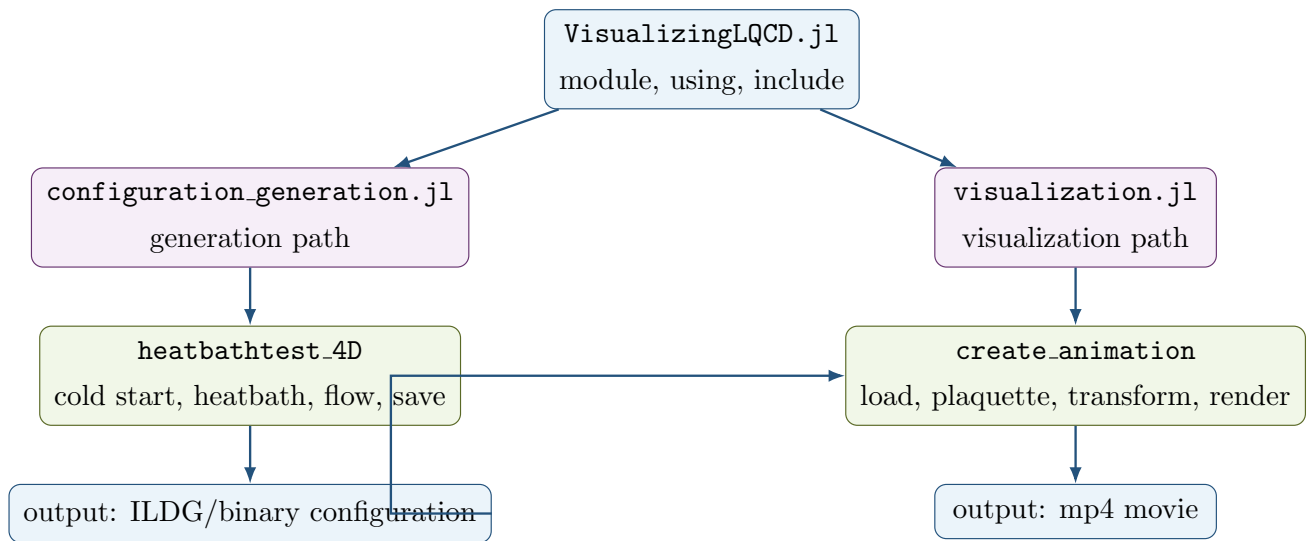


図 1 現行コードの大まかな実行時構造。生成 path と可視化 path は同じ module に入っているが、file 出力を介して接続される。

3.3 全関数の入出力型一覧

現行コードの多くの関数は、Julia の signature 上では型注釈を持たない。したがって、表では「宣言上の型」と「実際に期待される型」を分けて書く。宣言上の型がない引数は `Any` として受け取られるが、内部の配列確保、loop、Gaugefields API、Makie API により、実用上は `Int`、`Float64`、`String`、Gaugefields の field 型が期待される。戻り値についても、明示 `return` がない関数は side effect を主目的とし、戻り値に API 契約を置かない方が安全である。

表 2: 現行 source に定義される関数の I/O 型。

関数	入力	出力	副作用・注意点
<code>automatic_level2</code>	宣言上は <code>plaqs_t::Any</code> 。実用上は <code>AbstractArray{<:Real}</code> 。 <code>minimum</code> , <code>maximum</code> , <code>mean</code> , <code>mode</code> , <code>std</code> が定義されている必要がある。	<code>(level, isorange, min_val, max_val)</code> 。実用上は実数 tuple。 <code>level=mean(plaqs_t)</code> 、 <code>isorange=std(plaqs_t)</code> 。	統計量を <code>println</code> で表示する。空配列や非数値配列では失敗する。 <code>level selection</code> と診断表示が同じ関数に混ざっている。
<code>ln_a</code>	<code>beta</code> は厳密に <code>Float64</code> 。	<code>Float64</code> 。 $\log(a/r_0)$ の fit 値。	$\beta \notin [5.7, 6.57]$ で <code>ArgumentError</code> を投げる。 <code>Int</code> や <code>Float32</code> には method がない。
<code>calculate_a</code>	<code>beta</code> は厳密に <code>Float64</code> 。	<code>Float64</code> 。 $a = r_0 \exp(\ln a)$ を fm 単位で返す。	<code>ln_a</code> に依存するため、同じ β 範囲制限を持つ。

関数	入力	出力	副作用・注意点
<code>create_animation</code>	宣言上は全 positional 引数と keyword が型なし。実用上は <code>NX</code> , <code>NY</code> , <code>NZ</code> , <code>NT</code> , <code>NC::Int</code> , <code>videoname::AbstractString</code> , <code>beta::Float64</code> , <code>filename::AbstractString</code> 。	明示 return はない。最後の式は Makie の <code>record(...)</code> なので、戻り値は現在の Makie 実装に依存し、API 契約にしない方がよい。	ILDG file を読み、mp4 などを <code>videoname</code> に書く。GLMakie figure を作る。 <code>flow_steps_in</code> と <code>scale_factor</code> は現行関数内では使われない。 <code>levels[1]</code> が空 level に弱い。
<code>heatbath_SU3!</code>	宣言上は全引数が型なし。実用上、 <code>U</code> は 4 方向の Gaugefields gauge-link collection、 <code>NC::Int</code> , <code>temps</code> は長さ 5 以上で <code>similar(U[1])</code> 型の作業 field 群、 <code>β::Real</code> 。	<code>Nothing</code> と扱うのが自然。主出力は mutation された <code>U</code> 。	<code>U[μ]</code> を even/odd で破壊的に更新する。 <code>temps[5]</code> を staple 作業領域として使う。 <code>loops_staple</code> , <code>SU3update_matrix!</code> , <code>evaluate_gaugelinks_evenodd!</code> , <code>map_U!</code> に依存する。
<code>heatbathtest_4D</code>	宣言上は全引数が型なし。実用上は <code>NX</code> , <code>NY</code> , <code>NZ</code> , <code>NT</code> , <code>NC::Int</code> , <code>flow_steps_in::Int</code> , <code>β::Real</code> , <code>confname::AbstractString</code> 。	<code>plaq_t</code> 。実用上は最後に測った normalized plaquette の実数値。	cold start から配位を生成し、heatbath 20 sweeps, gradient flow, <code>save_binarydata(U, confname)</code> を実行する。標準出力に plaquette, Polyakov loop, progress を表示する。
<code>test()</code> in <code>test/runtests.jl</code>	入力なし。	明示 return はない。 <code>create_animation</code> の戻り値に依存しない smoke test と見るべきである。	$12^3 \times 16$ 配位を生成し、ILDG/binary file と mp4 を作る。GUI/backend と動画 encoder に依存するため通常 CI では重い。

3.4 局所関数と匿名関数

`heatbath_SU3!` の内部には、`mapfunc!` という局所関数があり、この関数が `SU3update_matrix!` を呼ぶ [5]。入力 `A` と `B` は Gaugefields の `map_U!` から渡される link/staple 型の値であり、`mapfunc!` は β 、 N_c 、作業行列、反復上限を closure として捕捉する。出力は `SU3update_matrix!` の戻り値に依存するが、実質的な主出力は `A` の破壊的更新である。

`create_animation` の内部には、`record(fig, videoname, 1:t_end; framerate=framerate)` `do i ... end` の匿名関数がある [4]。入力 `i` は frame index であり、現行コードでは `t = i % NT + 1` によって第 4 方向 slice を選ぶ。出力値は使われず、主な副作用は既存 contour object の削除、新しい contour の描画、title 更新、動画 frame の記録である。ここは off-by-one を直す PR 0 の直接対象であり、`slice4=(i-1)%NT+1` にすれば最初の frame が slice 1 になる。

3.5 top-level 定数と実行時副作用

`visualization.jl` は `include` 時に `GLMakie.activate!()` を実行し、さらに `LtoSec=10/3`、`myxlabel`、`myylabel`、`myzlabel`、`r_0=0.48` を top-level 定数として定義する [4]。このうち `LtoSec` は `yoctosecond` 表示を作るための値なので、時間表示廃止後は `legacy default` として隔離する。`r_0` と `ln_a` の多項式係数は物理単位換算に関わるため、`ScaleSpec` に移し、`metadata` に `source` と一緒に保存するのがよい。

`constants.jl` は、`NX=24`、`NY=24`、`NZ=24`、`NT=32`、 $\beta = 6.0$ 、`NC=3`、`flow_steps_in=200`、`confname`、`videoname` を top-level 変数として定義する [9]。現行 `module` からは使われていないため、PR では削除よりも `examples/` へ移す方が安全である。`header.jl` も同様に、`module` 本体が同じ依存を読むため、現行 `package API` には影響しない旧補助ファイルとして扱う [8]。

3.6 I/O 型から見える改修方針

この I/O 棚卸しから、最初に直すべき点は 3 つに絞れる。第一に、`create_animation` の戻り値を曖昧な `record` 依存にせず、`AnimationResult` または `NamedTuple` として、`video_path`、`metadata_path`、`levels`、`raw_equivalent_levels` を返す。第二に、`heatbathtest_4D` は `GenerationResult` を返し、`plaq_t` だけでなく `confname`、`final_plaquette`、`final_polyakov`、`flow_steps` を記録する。第三に、`automatic_level2` は表示をやめ、`LevelStats` か `named tuple` を返す純粋関数へ変える。

型注釈は過剰に付ける必要はないが、API 境界では最低限の `validation` を入れるべきである。格子サイズは正の整数、 N_c は正の整数、 β は `Real` を受けて内部で `Float64` へ変換する、入力 `file` は存在確認する、出力 `file` の拡張子と `backend` は整合性を確認する。この `validation` を入れるだけで、現行の「深い依存関数の中で失敗する」挙動を減らせる。

4 現行実装の詳細棚卸し

4.1 呼び出しグラフ

現行コードで `export` される関数は `create_animation` と `heatbathtest_4D` である。非 `export` の helper は `automatic_level2`、`ln_a`、`calculate_a`、`heatbath_SU3!` であり、`test/runtests.jl` には `package` 外の `test()` が定義される [4, 5, 6]。呼び出し関係は小さいが、`create_animation` の中に `I/O`、`observable`、`transform`、`level`、`renderer` が密結合している。したがって、関数数は少ないが、単体テスト可能な責務はかなり多い。

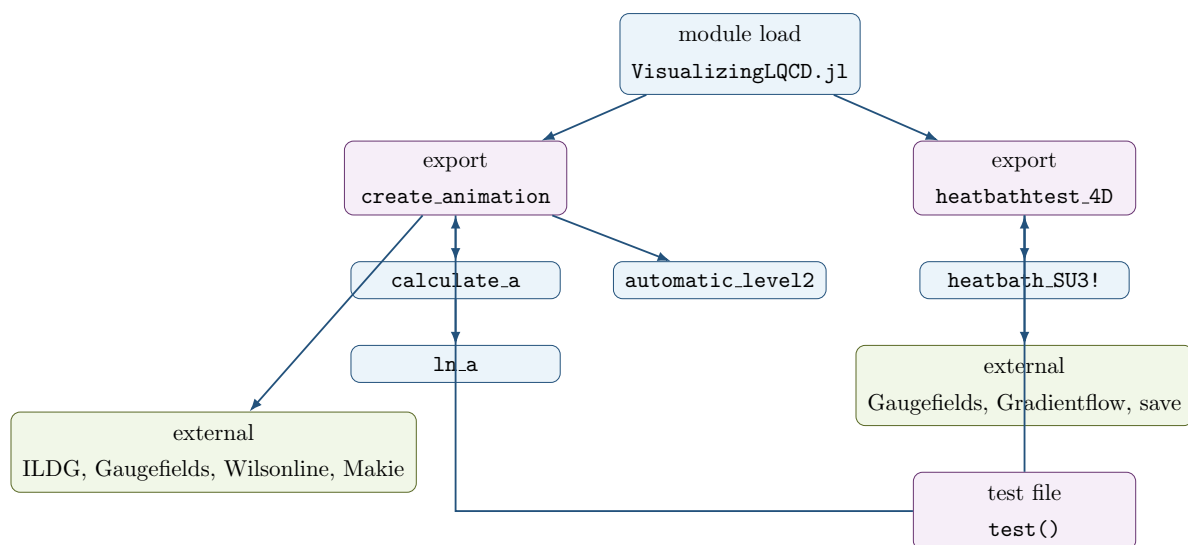


図 2 現行関数の呼び出しグラフ。関数数は少ないが、`create_animation` が多数の責務を同時に担う。

4.2 外部 API 依存の書き出し

現行関数の型は、依存 package の API によって実質的に決まっている。Initialize_Gaugefields はゲージ場の入れ物を作り、ILDG と load_gaugefield! は file から配位を読み、Wilsonian と evaluate_gaugelinks! はプラケット loop を評価する [4]。heatbathtest_4D 側では、calculate_Plaquette、calculate_Polyakov_loop、Gradientflow、flow!、save_binarydata が主要な外部 API である [5]。この依存表を作ると、切り出すべき境界が明確になる。

表 3: 現行コードが呼ぶ外部 API と分離先。

外部 API	現行での用途	依存元	分離後の置き場所
GLMakie.activate!()	GLMakie backend を有効化する。	top-level	renderers.jl。module include 時には実行しない。
Initialize_Gaugefields	gauge field container を作る。	create_animation, heatbathtest_4D	io.jl と generation.jl。
ILDG, load_gaugefield!	ILDG file を配位へ読む。	create_animation	io.jl。
Wilsonian	plaquette path を Wilson line として表す。	create_animation	observables.jl。
Gaugefields.evaluate_gaugelinks!	Wilson line に沿った link 積を全格子点で評価する。	create_animation	observables.jl。
tr, real	SU(3) plaquette matrix から scalar を取る。	create_animation	observables.jl。

外部 API	現行での用途	依存元	分離後の置き場所
mean, std, mode	変換後 field の統計量を出す。	automatic_level2	levels.jl または diagnostics.jl。
Figure, Axis3, contour!, record	3D 等値面と動画を作る。	create_animation	renderers.jl。
loops_staple	heatbath staple loop の定義を得る。	heatbath_SU3!	generation.jl。
evaluate_gaugelinks_evenodd!	even/odd site の staple を評価する。	heatbath_SU3!	generation.jl。
SU3update_matrix!, map_U!	SU(3) link を破壊的に更新する。	heatbath_SU3!	generation.jl。
calculate_Plaquette	平均 plaquette を測る。	heatbathtest_4D	generation.jl または measurements.jl。
calculate_Polyakov_loop	Polyakov loop を測る。	heatbathtest_4D	generation.jl または measurements.jl。
Gradientflow, flow!	gauge field を flow で滑らかにする。	heatbathtest_4D	flow.jl。
save_binarydata	配位を file へ保存する。	heatbathtest_4D	io.jl または generation.jl。

4.3 create_animation の内部変数と形状

create_animation は、可視化 pipeline の全段階を 1 関数にまとめている。内部変数の多くは後続 PR で独立した型に移せる。とくに U1、Uloop、temps は Gaugefields/Wilsonloop 側の作業データであり、plaqs_t は renderer へ渡すべき scalar field である。levels、x_physical、tick label、fig、ax、plot_obj は描画設定であり、observable の計算とは切り離せる。

表 4: create_animation の内部変数の棚卸し。

変数	推定型・形状	意味	分離先
Nwing	Int, 値 0	Gaugefields の wing 幅。	ConfigSpec。
a	Float64	β から推定した格子間隔 fm。	scales.jl。
scale_factor	Float64	現行では代入されるが未使用。	削除 または ScaleSpec。
U1	4 方向 gauge field collection	ILDG を読み込む入れ物。	io.jl。
ildg	ILDG reader object	入力 file handle または reader。	io.jl。
loop	Vector{Tuple{Int,Int}}	(1, +), (2, +), (1, -), (2, -) の plaquette path。	PlaquettePlane((1,2))。
w	Wilsonline object	loop を Wilsonloop.jl の表現へ変換したもの。	observables.jl。

変数	推定型・形状	意味	分離先
Uloop	similar(U1[1])	各点の plaquette matrix を保持する作業 field。	observables.jl。
temps	長さ 10 の作業 field vector	Wilsonline 評価の temporary buffer。	observables.jl 内部。
tmp	Float64	$1 - \Re \text{tr} U_{12}/N_c$ の raw plaquette deviation。	ScalarField.raw。
plaqst	Array{Float64,4}, (N_X, N_Y, N_Z, N_T)	$-\log(tmp + 10^{-7})$ 変換後 field。	ScalarField.display。
level	実数	変換後 field の平均。	LevelStats。
isorange	実数	変換後 field の標準偏差。	LevelStats。
levels	Vector{Float64}	等値面の表示レベル。	LevelSpec と LevelResult。
x_physical 等	Tuple{Float64,Float64}	Makie contour に渡す座標範囲。	SpatialAxes。
x_positions 等	StepRangeLen	tick 位置。	AxisSpec。
x_labels 等	Vector{String}	tick label。1 つおきに空文字へ変える。	AxisSpec。
fig	Makie Figure	描画 canvas。	renderers.jl。
ax	Makie Axis3	3D 軸。	renderers.jl。
dummy_data	Array{Float64,3}	最初の contour object を作るための空 field。	renderer 内部。
plot_obj	Makie plot object	frame ごとに消して再作成される contour。	renderer 内部。
framerate	Int, 値 12	動画 fps。	RenderSpec。
t_end	Int, 現行 NT*1	frame 数。	FrameSpec。
t	Int	第 4 方向 slice。現行は $i\%NT+1$ 。	frame_map。
t_phys	Float64/String	yoctosecond 表示用。廃止対象。	legacy metadata。

4.4 heatbath_SU3! と heatbathtest_4D の内部変数

配位生成側は、可視化本体から外しても document には残すべきである。heatbath_SU3! は破壊的更新関数であり、heatbathtest_4D はデモ用 driver である。生成側の変数は、将来の GenerationSpec、HeatbathSpec、FlowSpec、GenerationResult に移せる。

表 5: 配位生成側の内部変数の棚卸し。

変数	推定型・形状	意味	分離先
Dim	Int, 値 4	4 次元 Euclidean 格子。	LatticeSpec。
V	Gaugefields field	temps[5] を staple 作業領域として使用。	HeatbathWorkspace。
ITERATION_MAX	Int, 値 100000	SU3update.matrix! の反復上限。	HeatbathSpec。

変数	推定型・形状	意味	分離先
temps2	長さ 5 の 2×2 complex matrix vector	SU(2) subgroup 更新用の作業行列と推定される。	HeatbathWorkspace。
temps3	長さ 5 の $N_c \times N_c$ complex matrix vector	SU(3) 更新用の作業行列。	HeatbathWorkspace。
mapfunc!	局所関数	A, B を受けて SU3update_matrix! を呼ぶ closure。	heatbath_kernel!。
loops	Gaugefields loop object	loops_staple[(Dim,mu)] から得る staple path。	HeatbathSpec。
iseven	Bool	even/odd site の切替。	update loop 内部。
U	4 方向 gauge field collection	cold start で作った生成対象。	GenerationState。
temp1-temp5	similar(U[1])	plaquette、Polyakov、heatbath の作業 field。	GenerationWorkspace。
comb	Int, 値 6	4 次元の面数 $\binom{4}{2}$ 。	binomial(Dim,2)。
factor	実数	plaquette 正規化係数 $1/(6NVN_c)$ 。	PlaquetteNormalization。
plaq_t	実数	正規化 plaquette。戻り値。	GenerationResult。
poly	複素数	Polyakov loop。現行では表示だけ。	GenerationResult。
numhb	Int, 値 20	heatbath sweep 数。	HeatbathSpec。
g	Gradient flow object	Gradientflow(U) の戻り値。	FlowState。
flow_steps	Int	gradient flow の繰り返し回数。	FlowSpec。
filename	String	confname を代入した保存先。	GenerationResult。

4.5 関数ごとの失敗条件と validation

現行コードは、多くの前提を深い依存関数に任せている。利用者向け API にするなら、入口で検査した方が原因が分かりやすい。たとえば β の範囲外は `ln_a` が `ArgumentError` を投げるが、`file` が存在しない場合、ILDG の格子サイズと引数が合わない場合、`level` 列が空の場合、GLMakie backend が使えない場合は、より上流で明示した方がよい。

表 6: 現行関数の主な失敗条件と追加すべき validation。

対象	起こり得る失敗	追加すべき検査・対策
automatic_level2	空配列、NaN だけの配列、 <code>mode</code> が重みまたは非数値。	finite 値だけを抽出し、空なら明示 error。mode は任意診断へ移す。
ln_a	Float64 以外に method がない。範囲外で error。	Real を受けて内部で Float64 化。範囲外では lattice units fallback を選べるようにする。

対象	起こり得る失敗	追加すべき検査・対策
<code>calculate_a</code>	<code>ln_a</code> と同じ。	<code>ScaleSpec</code> に <code>source</code> 、範囲、 <code>fallback</code> を持たせる。
<code>create_animation</code>	入力 <code>file</code> がない。file 中の格子サイズと引数が違う。 N_c が違う。	file 存在確認、lattice shape 照合、 N_c 照合、読み込み後の sanity check。
<code>create_animation</code>	<code>levels</code> が空になり、 <code>levels[1]</code> で失敗。	level selection が空なら fallback level を作るか、明示 error を出す。
<code>create_animation</code>	<code>tmp<0</code> や丸め誤差で $-\log(\text{tmp}+\text{eps})$ が問題化する。	raw を $\max(\text{tmp}, 0)$ で扱うか、負値 count を metadata に保存する。
<code>create_animation</code>	GLMakie/encoder が headless 環境で失敗。	backend を <code>:auto</code> にし、CI は Cairo-Makie 静止画 smoke test にする。
<code>create_animation</code>	最初の frame が slice 2 になる。	<code>slice4=(i-1)%NT+1</code> へ変更し、unit test を追加。
<code>heatbath_SU3!</code>	<code>temps</code> が短い、 <code>temps[5]</code> が合わない、 $N_c \neq 3$ で SU3 専用関数と齟齬。	<code>length(temps)>=5</code> 、field shape、 $N_c = 3$ または実対応範囲を入口で明示。
<code>heatbathtest_4D</code>	格子サイズが非正、flow step が負、保存先 directory がない。	<code>GenerationSpec</code> で validate。保存先 directory を確認。
<code>test()</code>	動画生成が CI で重い、backend 依存で不安定。	通常 test と heavy integration test を分ける。

4.6 現行処理を段階名へ写した疑似コード

以下の疑似コードは、現行 `create_animation` の処理を段階名に置き換えたものである。現行の動作を保ったまま、分割後の関数名を与えよう。PR では、この順序を変えずに関数を分けると、回帰確認が容易になる。

Listing 1 現行 `create_animation` を段階化した疑似コード

```

1  function current_create_animation_pipeline(args)
2      # 1. Scale
3      a = calculate_a(beta)
4
5      # 2. I/O
6      U = allocate_gaugefield(NC, Nwing, NX, NY, NZ, NT)
7      load_configuration!(U, filename; lattice=(NX,NY,NZ,NT), nc=NC)
8
9      # 3. Observable
10     Uloop = evaluate_wilsonline(U, loop=[(1,+1),(2,+1),(1,-1),(2,-1)])
11     raw = Array{Float64}(undef, NX, NY, NZ, NT)
12     for x,y,z,t in lattice_sites
13         raw[x,y,z,t] = 1 - real(tr(Uloop[:, :, x,y,z,t])) / NC
14     end
15 
```

```

16     # 4. Display transform, current-compatible
17     display = -log.(raw .+ 1e-7)
18
19     # 5. Level selection, current-compatible
20     level = mean(display)
21     isorange = std(display)
22     levels = collect((level + 1.2 * isorange):0.05:maximum(display))
23
24     # 6. Rendering
25     axes = physical_axes(a, NX, NY, NZ)
26     frame_map = [(frame=i, slice4=(i % NT) + 1) for i in 1:NT]
27     render_makie_movie(display, axes, levels, frame_map, videoname)
28 end

```

4.7 推奨する戻り値契約

現行 `create_animation` は、動画 file を書く副作用を主目的にし、戻り値を明示しない。改善版では、処理結果を返す。戻り値があれば、動画を開かなくてもテストで level、frame map、metadata path を確認できる。配位生成側も同様に、最後の `plaque` だけでなく、保存 file、flow step、Polyakov loop、測定履歴を返す。

Listing 2 戻り値契約の型案

```

1  struct AnimationResult
2      video_path::String
3      metadata_path::String
4      lattice::NTuple{4,Int}
5      nc::Int
6      beta::Union{Nothing,Float64}
7      observable_name::String
8      transform_name::String
9      epsilon::Float64
10     display_levels::Vector{Float64}
11     raw_equivalent_levels::Vector{Float64}
12     frame_map::Vector{NamedTuple}
13 end
14
15 struct GenerationResult
16     configuration_path::String
17     lattice::NTuple{4,Int}
18     nc::Int
19     beta::Float64
20     heatbath_sweeps::Int
21     flow_steps::Int
22     final_plaquette::Float64

```

```

23     final_polyakov::ComplexF64
24 end

```

4.8 推奨する関数シグネチャー一覧

型を付けすぎると Julia の柔軟性を損なうため、内部関数は抽象型で受けてよい。ただし、public API と file 境界は明示する。以下は、v7 で提案する最小限の書き換え先である。

表 7: 現行関数から移行する推奨シグネチャ。

責務	推奨シグネチャ
設定 validation	<code>validate(spec::ConfigSpec)::Nothing</code>
格子間隔	<code>lattice_spacing_fm(beta::Real, scale::ScaleSpec)::Float64</code>
配位読み込み	<code>load_configuration(spec::ConfigSpec)::Any</code>
frame map	<code>make_frame_map(NT::Integer; loops::Integer=1)::Vector{NamedTuple}</code>
単一面 plaquette	<code>measure(U, obs::PlaquettePlane)::ScalarField</code>
全 plaquette 和	<code>measure(U, obs::PlaquetteSum)::ScalarField</code>
表示変換	<code>build_display_field(raw::ScalarField, spec::TransformSpec)::FieldBundle</code>
level 選択	<code>choose_levels(data::AbstractArray, spec::LevelSpec)::LevelResult</code>
逆変換	<code>invert_display_levels(levels, spec::TransformSpec, epsilon::Real)::Vector{Float64}</code>
描画	<code>render_movie(field::ScalarField, axes::SpatialAxes, levels::Vector, frames, spec::RenderSpec)::String</code>
metadata	<code>write_metadata(path::AbstractString, result::AnimationResult)::String</code>
統合 API	<code>visualize_lqcd(spec::ConfigSpec; kwargs...)::AnimationResult</code>
互換 wrapper	<code>create_animation(...; kwargs...)::AnimationResult</code>
heatbath 更新	<code>heatbath_SU3!(U, spec::HeatbathSpec, workspace)::Nothing</code>
配位生成	<code>generate_quenched(spec::GenerationSpec)::GenerationResult</code>

4.9 metadata に現行値から書き出せる項目

現行コードだけからでも、かなり多くの情報を metadata に書ける。重要なのは、動画の見た目を決めた値と、物理的解釈に必要な値を同時に残すことである。たとえば `loop`、`epsilon`、`levels`、`beta`、`a`、`r_0`、`framerate`、`figure size`、`colormap`、`frame map` はすべて現行関数内で分かる。したがって、metadata 追加は大きな構造変更なしでも PR 2 より前に入れられる。

Listing 3 現行 create-animation から直接書き出せる metadata 項目

```
{
  "lattice": [NX, NY, NZ, NT],
  "nc": NC,
  "beta": beta,
  "lattice_spacing_fm": a,
  "scale": {"r0_fm": 0.48, "beta_range": [5.7, 6.57]},
  "input_file": filename,
  "output_video": videoname,
  "observable": {
    "kind": "PlaquettePlane",
    "plane": [1, 2],
    "loop": [[1, 1], [2, 1], [1, -1], [2, -1]],
    "raw_formula": "1 - real(tr(Uloop))/NC"
  },
  "transform": {
    "kind": "neglog",
    "epsilon": 1e-7,
    "formula": "-log(raw + epsilon)",
    "raw_focus_for_upper_levels": "low_raw_deviation"
  },
  "levels": {
    "method": "mean_std_legacy",
    "mean": level,
    "std": isorange,
    "start": level + 1.2 * isorange,
    "step": 0.05,
    "display_values": levels,
    "raw_equivalent_values": [exp(-L) - 1e-7 for L in levels]
  },
  "render": {
    "backend": "GLMakie",
    "figure_size": [800, 800],
    "colormap": "viridis",
    "alpha": 1.0,
    "framerate": 12,
    "screen_time_label": false
  },
  "frame_map": [{"frame": i, "slice4": ((i - 1) % NT) + 1}]
}
```

4.10 最小修正で追加できる実装片

完全な分割を待たず、現行 `create_animation` に小さな関数を足すだけでも改善できる。以下の関数は `Gaugefields` に依存しないため、すぐに単体テストできる。最初にこれを入れると、時間表示廃止、frame order 修正、対数 level の解釈保存が同時に進む。

Listing 4 現行コードへ先に足せる純粋関数

```
1  make_frame_map(NT::Integer; loops::Integer=1) = [  
2      (frame=i, slice4=((i - 1) % NT) + 1) for i in 1:(NT * loops)  
3  ]  
4  
5  raw_equivalent_level_neglog(L::Real; epsilon::Real=1e-7) =  
6      max(exp(-Float64(L)) - Float64(epsilon), 0.0)  
7  
8  function legacy_mean_std_levels(display::AbstractArray; multiplier=1.2, step=0.05)  
9      values = collect(skipmissing(vec(display)))  
10     filter!(isfinite, values)  
11     isempty(values) && error("no finite display values")  
12     mu, sigma = mean(values), std(values)  
13     start = mu + multiplier * sigma  
14     stop = maximum(values)  
15     start > stop && return [stop]  
16     return collect(start:step:stop)  
17 end
```

5 現行コードのマジックナンバーを参考値として残す

5.1 基本方針

現行コードに含まれる固定値は、単に消すのではなく、移行時の比較基準として記録する。とくに $\epsilon = 10^{-7}$ 、 $\mu + 1.2\sigma$ から始める等値面レベル、level 刻み 0.05、framerate 12 は、既存動画の見え方を再現するために重要である [4]。v7 では、これらを「legacy default」または「reference value」として metadata と設定型に移し、意味が曖昧なハードコードを減らす。値そのものをすべて推奨値とみなすのではなく、互換 wrapper、回帰テスト、発表済み動画との比較に使う。

5.2 可視化経路の参考値

10^{-7} $-\log(\text{tmp} + 0.0000001)$ の ϵ である。ゼロ近傍を避け、対数変換後の閾値選択を安定化するため、`TransformSpec(:neglog, 1e-7, :fixed, ...)` の `legacy default` として残す。

1.2 σ と 0.05 現行コードは `mean + 1.2*std` から始め、0.05 刻みで `max` まで level を作る。新既定は分位点方式にするが、旧方式は `LevelSpec(:mean_std)` として残し、raw equivalent value と一緒に metadata へ保存する。

`automatic_level2`

`mean`, `mode`, `std`, `min`, `max` を表示する診断関数である。今後は histogram、分位点、voxel count、level 感度と合わせて metadata に残す。

`:neglog` 現行の `-log(tmp + eps)` は、上側 level が raw p の小さい側を強調する。既存互換の表示変換として名前を付け、metadata に `raw_focus="low_raw_deviation"` を保存する。

`xy plaque` `Wilsonline` は `[(1,+1),(2,+1),(1,-1),(2,-1)]` で、 xy 面の 1×1 プラケットを表す。この値は `PlaquePlane((1,2))` として observable 名へ移す。

作業配列 10 個

`temps = [similar(U1[1]) for _ in 1:10]` は `Wilsonline` 評価の作業領域である。必要数を関数化し、observable 実装の内部詳細として隠す。

`beta=6.1` `create_animation` の default であり、README 例の $\beta = 6.0$ とは異なる。新 API では β を明示入力にし、この値は互換 wrapper にだけ残す。

既定ファイル名

`filename="conf_00000100.ildg"` は現行 default である。新 API では入力ファイルを必須引数にし、互換 wrapper の fallback としてだけ残す。

`flow_steps_in=200`

`create_animation` の default だが、現行関数内では使われない。README の scratch 生成例では 200 steps を使うため、`FlowSpec.steps` へ移し、読み込み後に flow を行うかどうかを明示する。

5.3 描画と画面構成の参考値

`LtoSec=10/3` fm/c を yoctosecond へ換算する表示用値である。時間表示を止める方針により、画面からは廃止する。旧動画の再現に必要な場合だけ legacy metadata に残す。

`800 × 800` `Figure(size=(800,800))` の canvas サイズである。`RenderSpec` に移す。

色と透明度 `colormap=:viridis`, `transparency=false`, `alpha=1.0` は現行の描画 style である。`RenderStyle` に移し、metadata に保存する。

軸設定 `aspect=(1,1,1)`、tick label の `digits=2`、tick label を 1 つおきに消す `stride 2` は見た目の設定である。`AxisSpec` として名前を付ける。

動画設定 `framerate=12` と `t_end=NT*1` は、12 fps で第 4 方向を 1 周だけ再生する指定である。`RenderSpec` と `FrameSpec` に移す。

フレーム順序 現行の `t = i % NT + 1` は最初の frame が slice 2 になる。`slice4=(i-1)%NT+1` に直し、frame map を JSON sidecar に保存する。

時刻整形 `round(t_phys,digits=3)` と `lpad(...,6,'0')` は yoctosecond 表示のための整形である。時間表示の廃止に伴い legacy 扱いにする。

5.4 スケール設定と配位生成の参考値

$r_0 = 0.48 \text{ fm}$ 現行の物理単位換算で使う Sommer scale である。 `ScaleSpec(:r0,0.48)` として source と一緒に metadata に保存する。

β 範囲 `ln.a(beta)` の適用範囲は `[5.7,6.57]` である。範囲外では error にするか、lattice units 表示へ fallback するかを `ScaleSpec` で選ぶ。

3 次多項式係数

`ln.a(beta)` は `-1.6805, -1.7139, 0.8155, -0.6667` を使う。係数は `ScaleSpec` に閉じ込め、source とともに metadata へ保存する。

`Dim=4, Nwing=0`

4次元 Euclidean 格子と halo/wing 幅の設定である。Dim は `ConfigSpec.lattice` から導き、Nwing は I/O と generation の設定に残す。

初期条件 `condition="cold"` は配位生成デモの cold start を表す。 `GenerationSpec` に移す。

更新上限 `ITERATION_MAX=100000` は SU(3) 更新の反復上限である。 `HeatbathSpec` にする。

作業配列 5 個 `temps2` と `temps3` の長さ 5 は heatbath 更新の実装詳細である。更新アルゴリズム内へ閉じ込める。

`comb=6` 4次元での $\mu < \nu$ 面数である。固定値ではなく `binomial(Dim,2)` から計算する。

heatbath 20 sweeps

`numhb=20` はデモ用の sweep 数である。 `GenerationSpec` として名前を付ける。

表示間隔 5 sweeps

`itrj % 5 == 0` は plaquette と Polyakov loop の表示間隔である。 `GenerationSpec` に移す。

flow step size 0.01

コメントに `flow_time*0.01` とあり、flow step size が 0.01 であることを示唆している。 `FlowSpec` として明示する。

5.5 README と test の参考値

README の例は、 $24^3 \times 32$ 、 $\beta = 6.0$ 、 $N_c = 3$ 、scratch 生成では `flow_steps_in=200` を使う [1]。test は $12^3 \times 16$ 、 $\beta = 6.0$ 、 $N_c = 3$ 、`flow_steps_in=10` を使い、配位生成と動画生成をまとめて走らせる [6]。これらは production default ではなく、demo size と CI smoke size として名前を付ける。たとえば `ExampleSpec(:readme)` と `ExampleSpec(:ci_small)` を分けると、README、テスト、回帰比較の意図が混ざらない。

Listing 5 同梱 skeleton に入れた reference default の例

```

1  const CURRENT_LOG_EPSILON = 1e-7
2  const CURRENT_LEVEL_STD_MULTIPLIER = 1.2
3  const CURRENT_LEVEL_STEP = 0.05
4  const CURRENT_MOVIE_FRAMERATE = 12
5  const CURRENT_MOVIE_NLOOPS = 1
6  const CURRENT_FIGURE_SIZE = (800, 800)
7  const CURRENT_COLORMAP = :viridis
8  const CURRENT_ALPHA = 1.0
9  const CURRENT_BETA_ANIMATION_DEFAULT = 6.1
10 const CURRENT_RO_FM = 0.48
11 const CURRENT_BETA_RANGE = (5.7, 6.57)
12 const CURRENT_LN_A_COEFFS = (-1.6805, -1.7139, 0.8155, -0.6667)
13 const CURRENT_HEATBATH_SWEEPS = 20
14 const CURRENT_HEATBATH_REPORT_INTERVAL = 5
15 const CURRENT_FLOW_STEP_SIZE = 0.01

```

v7 の追加提案

現行コードの固定値は、`current_defaults.jl` と `metadata schema` にまとめる。旧動画を再現したい場合は `LegacyVisualDefaults()` を選び、研究用の新規動画では `QuantileLogDefaults()` などを選ぶ。値を明文化すれば、どの値が物理 convention で、どの値が表示 style で、どの値が単なる demo size かを利用者が判断できる。

6 時間表示を止める

6.1 物理的な理由

動画のフレームは第 4 方向スライスの走査であり、実時間の運動ではない。README の `real time` という表現と、コード中の `yoctosecond` タイトルは、見た目の説明としては直感的だが、発表や教材では誤解を生みやすい [1, 4]。したがって、画面には時間を出さない。フレーム番号と第 4 方向スライスの対応は `metadata` に保存し、動画の見た目からは実時間の印象を取り除く。

この変更は情報を捨てるものではない。frame map を JSON sidecar に残せば、動画の第 i フレームがどの lattice slice に対応するかは復元できる。再現性は `metadata` が担い、画面表示は物理的に誤読されにくい表現にする。

Listing 6 時間表示を削る最小修正の骨子

```

1  # Remove LtoSec and yoctosecond conversion from the display path.
2  static_title = "Plaquette log iso-surface"
3  record(fig, videoname, 1:t_end; framerate=movie_framerate) do i
4      slice4 = (i - 1) % NT + 1

```

```

5     delete!(ax, plot_obj)
6     field3 = @view field4[:, :, :, slice4]
7     # Keep the title static. No time or slice label is shown by default.
8     ax.title = static_title
9     plot_obj = GLMakie.contour!(ax, x_physical, y_physical, z_physical, field3;
10                                levels=levels, colormap=:viridis,
11                                transparency=false, alpha=1.0)
12 end

```

6.2 表示と metadata の分担

画面仕様は単純にする。既定タイトルは、Plaqueette log iso-surface のように observable と transform だけを示す。フレーム番号、第 4 方向スライス、flow step、 β 、格子サイズ、 ϵ 、level は、動画ではなく JSON sidecar に保存する。必要な利用者には `show_frame_label=true` を用意してよいが、既定値は `false` にする。

7 対数変換を閾値座標として設計する

7.1 raw 量と表示変換を分ける

ブラケット deviation

$$p(x) = 1 - \frac{1}{N_c} \text{Re tr } U_{\mu\nu}(x) \quad (2)$$

は、格子間隔が小さい極限で $F_{\mu\nu}^2$ 型の局所量に対応する非負スカラーである。Wilson の格子ゲージ理論は、リンク変数のゲージ不変な閉路積から作用を構成する標準的な枠組みを与える [14]。しかし、等値面描画で raw p をそのまま閾値座標に使う必要はない。可視化では、 p から単調変換 $f(p)$ を作り、その f 空間で等値面レベルを決める方が安定な場合がある。

対数は、乗法的な変化を加法的な変化へ写す。たとえば raw 値が 2 倍変わると、 $\log p$ は $\log 2$ だけ変わる。これは、plaquette deviation の分布が細い本体と長い尾を持つとき、または gradient flow の step 数で値域が大きく変わるときに役立つ。したがって、対数は「物理量の定義」ではなく「閾値を置く座標系」である。

v7 の解釈

$-\log(p + \epsilon)$ は、等値面の閾値選択を安定化する display transform である。raw 物理量 p は保持し、変換後の $f(p)$ でレベルを選び、選んだレベルを raw p の閾値へ逆変換して metadata に保存する。この分離により、見やすさと物理的な説明を両立できる。

7.2 符号の向きを仕様に入れる

$\ell_+(x) = \log(p(x) + \epsilon)$ は、 p が大きいほど大きい。上側分位点で等値面を取ると、raw plaque deviation の大きい領域が強調される。一方、 $\ell_-(x) = -\log(p(x) + \epsilon)$ は、 p が小さいほど大きい。上側分位点で等値面を取ると、raw plaque deviation の小さい領域が強調される。

この違いを曖昧にすると、動画の解釈を誤る。v7 では、transform に `:log` と `:neglog` を用意し、metadata に `raw_focus` を記録する。既存動画の見え方を保つ場合は `:neglog` を既定にできるが、その時は `raw_focus="low_raw_deviation"` と明示する。強い plaque deviation の hot spot を見せたい場合は `:log` または `:identity` を選ぶ。

変換	表示値	上側レベルの意味	推奨説明
<code>:identity</code>	p	raw p が大きい領域	plaque deviation iso-surface
<code>:log</code>	$\log(p + \epsilon)$	raw p が大きい領域	log plaque-deviation iso-surface
<code>:neglog</code>	$-\log(p + \epsilon)$	raw p が小さい領域	negative-log plaque-deviation iso-surface

7.3 ϵ を隠さない

現状の 10^{-7} は、ゼロや極小値に対する数値的な保護として使われている [4]。ただし、 p が ϵ に近い領域では、 $\log(p + \epsilon)$ の値が ϵ に敏感になる。したがって、 ϵ はハードコードされた内部定数ではなく、TransformSpec の明示フィールドにする。

互換性を重視するなら既定値は 10^{-7} にする。研究発表用には、固定値を使うか、正の p の低分位点から自動決定する。自動決定を使う場合も、最終的に採用した ϵ を metadata に保存する。

Listing 7 対数変換を明示する型案

```

1  abstract type AbstractTransform end
2
3  struct TransformSpec <: AbstractTransform
4      kind::Symbol          # :identity, :log, :neglog, :zscore
5      epsilon::Float64      # default 1e-7 for current compatibility
6      epsilon_policy::Symbol # :fixed or :auto_quantile
7      clip_quantiles::Tuple{Float64,Float64}
8  end
9
10 function display_transform(raw::AbstractArray, spec::TransformSpec)
11     p = clamp.(raw, 0.0, Inf)

```

```

12     eps = choose_epsilon(p, spec)
13     if spec.kind == :identity
14         return p, eps, "raw"
15     elseif spec.kind == :log
16         return log.(p .+ eps), eps, "log(raw + epsilon)"
17     elseif spec.kind == :neglog
18         return .-log.(p .+ eps), eps, "-log(raw + epsilon)"
19     else
20         error("unknown transform kind: $(spec.kind)")
21     end
22 end

```

7.4 level の逆変換を必ず残す

変換後レベル L は raw p へ戻せる。 $\ell_+ = \log(p + \epsilon)$ なら、

$$p_{\text{level}} = \exp(L) - \epsilon \quad (3)$$

である。 $\ell_- = -\log(p + \epsilon)$ なら、

$$p_{\text{level}} = \exp(-L) - \epsilon \quad (4)$$

である。これを保存すれば、動画で使った等値面が raw plaquette deviation のどの閾値に対応したかを後から確認できる。

Listing 8 レベル逆変換の純粋関数案

```

1  function invert_display_level(level::Real, spec::TransformSpec, epsilon::Real)
2      if spec.kind == :identity
3          return Float64(level)
4      elseif spec.kind == :log
5          return max(exp(level) - epsilon, 0.0)
6      elseif spec.kind == :neglog
7          return max(exp(-level) - epsilon, 0.0)
8      else
9          error("level inversion is not defined for $(spec.kind)")
10     end
11 end

```

8 閾値選択を安定化する

8.1 分位点を既定にする

現状コードは、変換後データの平均と標準偏差からレベル列を作る [4]。この方法は簡単だが、外れ値、非対称分布、flow step の違いに敏感である。既定は分位点にする。たとえば $q = 0.80, 0.90, 0.95, 0.98$ を使えば、「変換後値の上位 20%、10%、5%、2% 付近の面」という意味が残る。

分位点は、可視化の比較にも向いている。格子サイズや β 、flow step、observable が変わっても、面の体積分率に近い意味を保てる。絶対値レベルが必要な場合は `:fixed` を選び、raw equivalent value を併記する。

Listing 9 level selection の型案

```
1 struct LevelSpec
2     method::Symbol          # :quantile, :fixed, :mean_std, :target_fraction
3     quantiles::Vector{Float64}
4     values::Vector{Float64}
5     tail::Symbol            # :upper, :lower, :explicit
6     global_across_slices::Bool
7 end
8
9 function choose_levels(display_data, spec::LevelSpec)
10     values = collect(vec(display_data))
11     filter!(isfinite, values)
12     isempty(values) && error("display field has no finite values")
13
14     if spec.method == :quantile
15         return unique(sort(quantile(values, spec.quantiles)))
16     elseif spec.method == :fixed
17         return unique(sort(spec.values))
18     elseif spec.method == :mean_std
19         mu, sigma = mean(values), std(values)
20         sigma == 0 && return [mu]
21         return collect((mu + 1.2sigma):0.05:maximum(values))
22     else
23         error("unknown level-selection method: $(spec.method)")
24     end
25 end
```

8.2 global level を既定にする

動画の各フレームで別々に分位点を計算すると、等値面の量が一定に見えやすいが、フレーム間の絶対的な強弱が消える。現状コードは 4 次元配列全体からレベルを決めるので、この点では global level になっている [4]。v7 でも既定は `global_across_slices=true` とする。

一方で、教材やデモでは各フレームの構造を見せたい場合がある。その場合は local level を選べるようにする。ただし、metadata に `global_across_slices=false` と明記し、フレームごとの level 値を保存する。これにより、動画の比較可能性を後から判断できる。

8.3 閾値感度を診断する

閾値に敏感な可視化では、レベルを少し変えるだけで等値面が大きく変わる。そこで、各 level L について、 $V(L) = |\{x : f(x) \geq L\}|$ を保存する。さらに、 $L \pm \Delta L$ での変化率

$$S(L) = \frac{|V(L + \Delta L) - V(L - \Delta L)|}{V(L) + 1} \quad (5)$$

を記録する。 $S(L)$ が大きい level は、見た目が閾値に強く依存している。

閾値安定性の実装方針

動画生成時に、raw と display の histogram、分位点、level ごとの voxel count、level 感度 $S(L)$ を JSON sidecar に保存する。これにより、対数変換を入れた理由を定量的に説明できる。実装は renderer から独立した `levels.jl` の純粋関数でよい。

9 部品独立性を高めるアーキテクチャ

9.1 責務分離の原則

現状の `create_animation` は、I/O、observable 計算、対数変換、level 決定、Makie 描画、動画記録を 1 つの関数内で行う。これはデモとしては良いが、改良するときに変更範囲が広くなる。v7 では、次の原則を採用する。

依存関係の原則

- observable は Makie を知らない。
- transform は Gaugefields を知らない。
- level selector は物理量の計算方法を知らない。
- renderer は ILDG とヒートバス更新を知らない。
- metadata writer は renderer の backend を知らない。
- configuration generation は visualization の必須経路に入らない。

9.2 推奨ファイル構造

小さい package なので、最初から別 package に分ける必要はない。まずは module 内でファイルを分割する。`src/VisualizingLQCD.jl` は全体を include するだけにし、重い backend は可能なら extensions や遅延読み込みへ寄せる。

Listing 10 推奨する src 構造

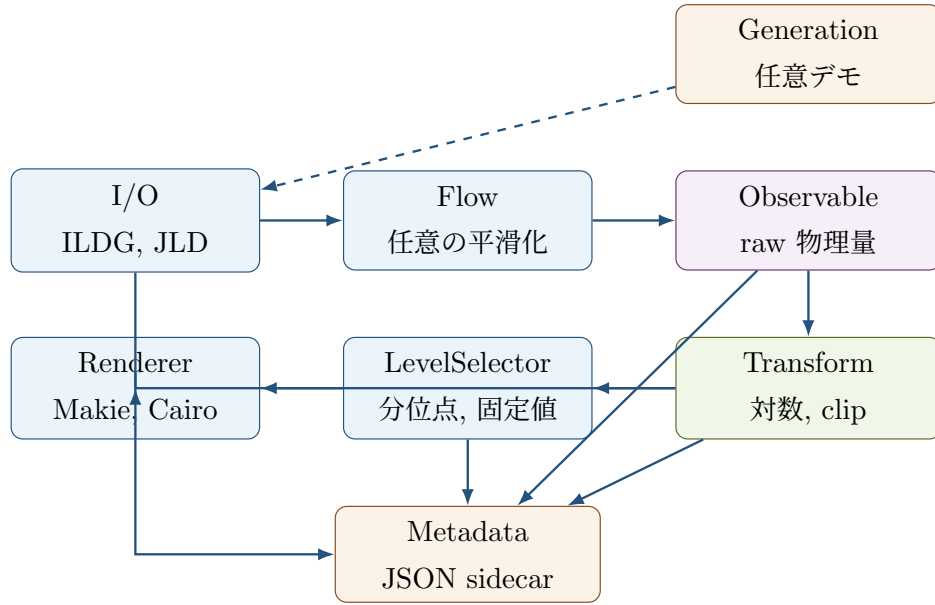


図 3 提案する依存関係。生成は入力例を作る部品であり、可視化 pipeline の必須部分ではない。

```

1  src/
2  VisualizingLQCD.jl
3  types.jl          # ConfigSpec, FlowSpec, TransformSpec, LevelSpec,
   RenderSpec
4  io.jl             # configuration loading and metadata extraction
5  scales.jl         # beta -> a, coordinate helpers, lattice-unit fallback
6  flow.jl           # optional gradient flow
7  observables.jl    # raw plaquette, clover energy, topology
8  transforms.jl     # identity, log, neglog, clipping, zscore
9  levels.jl         # quantile/fixed/mean_std/target_fraction levels
10 frames.jl         # slice iterator and frame map, no screen time labels
11 renderers.jl      # Makie/CairoMakie/GLMakie rendering
12 metadata.jl       # JSON sidecar and level reports
13 animation.jl      # high-level orchestration
14 configuration_generation.jl # optional demos

```

9.3 中核データ型

renderer には、Gaugefields のリンク配列ではなく、軽い scalar field を渡す。こうすると、renderer のテストは合成データだけで実行できる。observable の計算は Gaugefields と Wilsonloop に依存してよいが、transform と level selection は純粋な配列操作として保つ。

Listing 11 中核データ型のスケッチ

```

1  struct ConfigSpec
2      lattice::NTuple{4,Int}

```

```

3     nc::Int
4     beta::Union{Nothing,Float64}
5     filename::String
6 end
7
8 struct ScalarField{T,N,A<:AbstractArray{T,N}}
9     data::A
10    name::String
11    unit::String
12    description::String
13 end
14
15 struct FieldBundle{R,D}
16     raw::R
17     display::D
18     transform_name::String
19     epsilon::Float64
20     raw_focus::String
21 end
22
23 struct RenderSpec
24     backend::Symbol          # :glmakie, :cairomakie, :auto
25     output::String
26     framerate::Int
27     show_frame_label::Bool   # default false
28     title::String
29 end

```

10 高水準 API と互換性

10.1 新しい API

新 API は、設定をまとめた小さな型を受け取り、処理の各段階を明示する。既定値は、現状の見え方を大きく壊さないように `:neglog` を採用する。ただし、`metadata` で `raw_focus` を明示する。

Listing 12 高水準 API 案

```

1 visualize_lqcd(
2     ConfigSpec((24, 24, 24, 32), 3, 6.0, "Conf24242432beta6.0.ildg");
3     flow=FlowSpec(0, 0.01),
4     observable=PlaquettePlane((1, 2)),
5     transform=TransformSpec(:neglog, 1e-7, :fixed, (0.0, 1.0)),
6     levels=LevelSpec(:quantile, [0.80, 0.90, 0.95, 0.98], Float64[], :upper, true),
7     render=RenderSpec(:glmakie, "qcd_vacuum.mp4", 12, false,
8         "Plaquette log iso-surface"),

```

```

9     metadata="qcd_vacuum.json",
10 )

```

10.2 既存 create_animation の扱い

既存利用者を壊さないため、create_animation は残す。ただし、内部では visualize_lqcd を呼ぶ wrapper にする。現状の flow_steps_in は create_animation 内で使われていないので、wrapper では「読み込み後に任意で flow をかける step 数」として意味を統一する。時間表示は wrapper でも標準で消す。

Listing 13 互換 wrapper の骨子

```

1  function create_animation(NX, NY, NZ, NT, NC, videoname;
2      beta=6.1, flow_steps_in=0,
3      filename="conf_00000100.ildg")
4      spec = ConfigSpec((NX, NY, NZ, NT), NC, beta, filename)
5      return visualize_lqcd(spec;
6          flow=FlowSpec(flow_steps_in, 0.01),
7          observable=PlaquettePlane((1, 2)),
8          transform=TransformSpec(:neglog, 1e-7, :fixed, (0.0, 1.0)),
9          levels=LevelSpec(:quantile, [0.80, 0.90, 0.95, 0.98], Float64[], :upper,
10             true),
11          render=RenderSpec(:glmakie, videoname, 12, false,
12             "Plaquette log iso-surface"),
13          metadata=replace(videoname, r"\.[^.]+\$" => ".json"),
14      )
15  end

```

11 物理量の拡張

11.1 単一面ブラケットを互換 observable として残す

現状の xy 面ブラケットは、既存動画との互換性があり、局所構造を見せるには分かりやすい。したがって、PlaquettePlane((1,2)) として残す。これは「field strength 全体」ではなく、特定向きのブラケット由来スカラーであると説明する。

11.2 全 6 面ブラケット平均を既定候補にする

研究用の説明では、単一面より全 6 面平均の方が自然である。

$$s_P(x) = \frac{1}{6} \sum_{\mu < \nu} \left(1 - \frac{1}{N_c} \operatorname{Re} \operatorname{tr} U_{\mu\nu}(x) \right). \quad (6)$$

この量は厳密な clover energy density ではないが、向き依存を減らした Wilson 作用密度 proxy として説明できる。API では `PlaquetteSum()` を追加し、`PlaquettePlane((1,2))` と切り替えられるようにする。

11.3 Clover energy density と topological charge density

発表で「gluon energy density」を言う場合は、clover $F_{\mu\nu}$ から

$$E(x) = \frac{1}{4} \sum_{\mu, \nu, a} F_{\mu\nu}^a(x) F_{\mu\nu}^a(x) \quad (7)$$

を作る方が説明しやすい。topological charge density は

$$q(x) = \frac{1}{32\pi^2} \epsilon_{\mu\nu\rho\sigma} \text{tr}\{F_{\mu\nu}(x) F_{\rho\sigma}(x)\} \quad (8)$$

で定義でき、正負を別々の等値面で表示できる。QCDMeasurements.jl は plaquette、Polyakov loop、topological charge、energy density、Wilson loop などの測定機能を掲げているため、将来的には同 package の定義と比較するテストを入れるのが自然である [11]。

11.4 Gradient flow を表示 observable の一部として記録する

Gradient flow は、見た目を滑らかにするだけでなく、表示する場の定義の一部である。Lüscher の Wilson flow は、正の flow time で滑らかなゲージ場を与え、局所ゲージ不変量を扱いやすくする [13]。動画画面には時間を出さないが、metadata には flow step、積分刻み、可能なら t/a^2 と smoothing radius $\sqrt{8t}$ を残す。

12 metadata schema

12.1 JSON sidecar の最小要件

動画ファイルだけでは、どの物理量をどの変換で見せたかが分からない。v7 では、動画と同じ basename の JSON sidecar を出す。最低限、次の情報を保存する。

Listing 14 metadata sidecar の例

```
{
  "schema_version": 1,
  "interpretation": {
    "spacetime": "Euclidean lattice configuration",
    "screen_time_label": false,
    "not_real_time_minkowski_evolution": true
  },
  "configuration": {
    "filename": "Conf24242432beta6.0.ildg",
```

```

    "lattice": [24, 24, 24, 32],
    "nc": 3,
    "beta": 6.0,
    "source_format": "ILDG"
  },
  "frame_map": [
    {"frame": 1, "slice4": 1},
    {"frame": 2, "slice4": 2}
  ],
  "observable": {
    "raw_name": "plaquette_deviation",
    "definition": "1 - Re tr U_12 / Nc",
    "plane": [1, 2]
  },
  "display_transform": {
    "kind": "neglog",
    "formula": "-log(raw + epsilon)",
    "epsilon": 1.0e-7,
    "epsilon_policy": "fixed",
    "raw_focus_for_upper_levels": "low_raw_deviation"
  },
  "level_selection": {
    "method": "quantile",
    "global_across_slices": true,
    "quantiles": [0.80, 0.90, 0.95, 0.98],
    "display_values": [7.1, 7.8, 8.4, 9.2],
    "raw_equivalent_values": [8.2e-4, 4.1e-4, 2.2e-4, 1.0e-4]
  }
}

```

12.2 タイトルと metadata の対応

動画タイトルは短く保つ。詳細は JSON に残す。たとえばタイトルは `Plaquette log iso-surface` とし、metadata に `kind="neglog"`、`raw_focus_for_upper_levels="low_raw_deviation"` を入れる。これにより、画面を情報過多にせず、解釈の根拠は消さない。

13 renderer と backend の独立性

13.1 Renderer は配列と座標だけを見る

Renderer は、`ScalarField` と `SpatialAxes` と `level` の配列だけを受け取る。Gaugefields のリンク更新、Wilsonline 評価、ILDG 読み込みを知らない。この設計により、renderer の smoke test は合成 field で実行できる。

Listing 15 Renderer 入力の型案

```

1  struct SpatialAxes
2      x::Vector{Float64}
3      y::Vector{Float64}
4      z::Vector{Float64}
5      xlabel::String
6      ylabel::String
7      ylabel::String
8  end
9
10 function render_movie(field::ScalarField, axes::SpatialAxes,
11                       levels::Vector{Float64}, spec::RenderSpec)
12     # Backend-specific implementation lives here.
13     # This function does not know how the field was measured.
14 end

```

13.2 GLMakie と CairoMakie を分ける

GLMakie は対話的な 3D 描画に向いているが、CI や headless server では不安定になりやすい。静止画 smoke test は CairoMakie、対話的表示と mp4 は GLMakie に分ける。backend=:auto の場合は、display の有無を見て backend を選ぶ。これにより、通常テストは GUI に依存しない。

14 I/O と配位生成の分離

14.1 I/O は配位を返すだけにする

io.jl は、ファイル名、格子サイズ、 N_c 、 β を受け取り、Gaugefields の配位を返す。ILDG 標準は、ゲージ配位のメタデータを XML schema で扱い、バイナリ配置と packing format を仕様化している [12]。可視化側では、その情報を可能な範囲で metadata に引き継ぐ。

14.2 配位生成は examples または submodule へ移す

heatbathtest_4D はデモとして価値がある。だが、可視化 API の必須経路に置くと、可視化だけを使う人にも生成アルゴリズムの依存が見える。生成は examples/generate_quenched.jl または VisualizingLQCD.Generation に移す。using VisualizingLQCD では、読み込み、observable、transform、level、rendering が主役になるようにする。

15 テスト計画

15.1 軽量単体テスト

部品分割の効果は、GUI なしで多くのテストが通ることである。現状の test は、 $12^3 \times 16$ の配位生成と動画生成を走らせる統合テストに近い [6]。v7 では、まず純粋関数のテストを厚くする。

分類	テスト内容	期待する効果
時間非表示	<code>show_frame_label=false</code> でタイトルに <code>yoctosecond</code> 、 <code>t</code> 、 <code>slice</code> が出ない。	誤読の再発を防ぐ。
フレーム順序	$NT = 4$ で frame map が 1, 2, 3, 4 になる。	off-by-one を防ぐ。
恒等配位	恒等リンクで <code>plaquette deviation</code> が 0 になる。	observable の基本を保証する。
対数変換	<code>:log</code> と <code>:neglog</code> が有限値を返し、逆変換で <code>raw level</code> が戻る。	閾値解釈を保証する。
ϵ	固定値と <code>auto</code> 値が <code>metadata</code> に保存される。	隠れた定数をなくす。
レベル選択	<code>NaN</code> 、定数配列、外れ値を含む配列で <code>level</code> が壊れない。	可視化の安定性を上げる。
ゲージ不変性	ランダムゲージ変換前後で <code>scalar observable</code> が一致する。	物理量としての正しさを確認する。
Renderer smoke	<code>synthetic field</code> を <code>CairoMakie</code> で 1 枚保存する。	GUI なし CI を可能にする。
metadata	<code>raw value</code> 、 <code>transform</code> 、 <code>level</code> 、 <code>frame map</code> が JSON に入る。	再現性を保証する。

15.2 重い統合テストを分ける

配位生成、`gradient flow`、`mp4` エンコードを含むテストは重い。通常 CI では実行せず、`nightly` または手動テストへ分ける。通常 CI は、`observables.jl`、`transforms.jl`、`levels.jl`、`metadata.jl`、`CairoMakie` の静止画 smoke test までにする。

16 段階的 PR 計画

16.1 PR 0: 表示の誤解をなくす

最初の PR は小さくする。`LtoSec` と `yoctosecond` への換算を削除し、タイトルを固定する。`t=i%NT+1` は `slice4=(i-1)%NT+1` に直す。README の `real time` 表現を削除し、`Euclidean slice`

sequence と説明する。対数変換は消さない。

16.2 PR 1: 対数変換と level selection を部品化する

TransformSpec、LevelSpec、apply_transform、invert_display_level、choose_levels を追加する。既定は現状互換の :neglog, epsilon=1e-7 とするが、metadata に raw_focus を保存する。レベル選択は分位点を既定にする。

16.3 PR 2: metadata sidecar を追加する

JSON sidecar を出す。frame map、observable、transform、 ϵ 、level、raw equivalent value、flow 情報を保存する。動画そのものは簡潔に保ち、意味は sidecar が担う。

16.4 PR 3: pipeline を分割する

io.jl、observables.jl、transforms.jl、levels.jl、renderers.jl、metadata.jl、animation.jl へ分割する。create_animation は互換 wrapper にする。renderer のテストは synthetic field で行う。

16.5 PR 4: observable を増やす

PlaquettePlane((1,2)), PlaquetteSum(), CloverEnergyDensity(), TopologicalChargeDensity() を順に追加する。QCDMeasurements.jl と同じ定義で値が合うかを比較するテストを入れる。

優先度	変更	完了条件	PR
P0	時間表示を外す。	タイトルに yoctosecond、 t 、slice が出ない。	0
P0	frame order を直す。	$NT = 4$ で 1, 2, 3, 4 になる。	0
P0	対数変換を温存する。	$-\log(p + 10^{-7})$ が transform として名前を持つ。	1
P0	level を分位点化する。	global quantile level が既定になる。	1
P1	JSON sidecar を出す。	raw equivalent level と frame map が復元できる。	2
P1	部品を分割する。	observable、transform、level、renderer が独立テストできる。	3
P2	全 6 プラケット平均を追加する。	PlaquettePlane と PlaquetteSum を切り替えられる。	4

P2	clover energy と topology を追加する。	QCDDMeasurements.jl と比較できる。	4
P3	backend を分ける。	GUI なし CI とローカル動画生成を分離できる。	3

17 リスクと対策

主要リスク		
リスク	内容	対策
:neglog の誤読	上側 level が raw p の小さい側を強調する。	<code>raw_focus</code> と <code>raw_equivalent_level</code> を <code>metadata</code> に入れる。
動画の見た目が変わる	分位点 level は旧 mean/std level と面の量が違う。	旧方式を <code>:mean_std</code> として残す。
ϵ 依存	小さい raw 値で display 値が ϵ に依存する。	ϵ と policy を保存し、 <code>auto</code> と <code>fixed</code> を切り替える。
CI が壊れる	GLMakie と動画エンコードが <code>headless</code> で失敗する。	CairoMakie smoke test と重い統合テストを分ける。
依存が重い	可視化だけで生成や backend が読まれる。	submodule または extension で遅延読み込みする。
物理説明が曖昧	field strength 全体と単一 plaquette plane が混同される。	observable 名を <code>PlaquettePlane((1,2))</code> とする。

18 コードスケルトンの内容

同梱 zip の `code_skeleton` には、完全実装ではなく、部品境界を示すための Julia ファイルを入れた。`transforms.jl` と `levels.jl` は、Gaugefields なしで読める純粋関数を中心に行っている。`animation.jl` は `orchestration` の形を示す。これを起点に、PR 1 と PR 2 を小さく実装できる。

Listing 16 orchestration の骨子

```

1  function visualize_lqcd(spec::ConfigSpec;
2      flow=FlowSpec(0, 0.01),
3      observable=PlaquettePlane((1, 2)),
4      transform=TransformSpec(:neglog, 1e-7, :fixed, (0.0, 1.0)),
5      levels=LevelSpec(:quantile, [0.8,0.9,0.95,0.98],
6                          Float64[], :upper, true),
7      render=RenderSpec(:glmakie, "qcd_vacuum.mp4", 12,
8                          false, "Plaquette log iso-surface"),
9      metadata="qcd_vacuum.json")
10  U = load_configuration(spec)
11  apply_gradient_flow!(U, flow)
12  raw = measure(U, observable)
13  bundle = build_display_field(raw, transform)
14  iso_levels = choose_levels(bundle.display.data, levels)
15  axes = spatial_axes(spec)
16  render_movie(bundle.display, axes, iso_levels, render)
17  write_metadata(metadata, spec, flow, observable, transform, levels,
18                render, iso_levels, bundle)
19  return bundle
20 end

```

19 結論

本提案の要点は、対数変換を消すのではなく、責務を明確にして残すことである。raw plaquette deviation p は物理量として保持し、 $-\log(p + \epsilon)$ は閾値を置くための表示座標として扱う。変換後の分位点で level を決め、raw equivalent value を metadata に保存すれば、見た目の安定性と物理的な説明が両立する。

時間表示は標準で止める。動画は Euclidean 配位のスライス列であり、Minkowski 実時間発展ではない。画面に yoctosecond や t を出さず、frame map は JSON sidecar に保存する。これにより、動画の直感性を保ちながら誤解を避けられる。

部品の独立性は、今後の改良速度を決める。I/O、flow、observable、transform、level selection、renderer、metadata、配位生成を分ければ、対数変換、物理量、backend、テストを別々に改善できる。VisualizingLQCD.jl は、既存の印象的なデモを保ちながら、再現性と説明可能性を備えた LQCD 可視化ツールへ進められる。

付録 A README 追記案

Listing 17 README 追記の下書き

```
1 Physics note:
2 The animation shows a sequence of three-dimensional slices of a Euclidean
3 lattice QCD configuration. It is not a real-time Minkowski evolution. For this
4 reason, the default movie title does not display a physical time or a slice
5 index. The slice-to-frame map is stored in the JSON metadata file.
6
7 The default visualized field is a log-transformed plaquette-derived scalar,
8 for example  $-\log(p + \epsilon)$ , where  $p = 1 - \text{Re tr } U_p / N_c$ . The logarithm is
9 not a replacement of the raw observable. It is a display transform used to
10 stabilize iso-surface threshold selection. The raw observable, transform,
11 epsilon, flow information, selected levels, and raw-equivalent level values are
12 stored in the metadata file.
```

付録 B 受け入れ基準チェックリスト

- 動画タイトルに yoctosecond、 t 、 τ 、slice index が表示されない。
- frame map が JSON sidecar に保存される。
- raw plaquette deviation と display transform が metadata で分かれている。
- $-\log(p + \epsilon)$ が `TransformSpec(:neglog, ...)` として明記される。
- ϵ とその決定方法が metadata に保存される。
- 変換後 level と raw equivalent level が metadata に保存される。
- `:neglog` の上側 level が low raw deviation を意味することが metadata に入る。
- $NT = 4$ のフレーム順序が 1, 2, 3, 4 である。
- global level が既定で、local level は明示選択である。
- `observables.jl` は Makie に依存しない。
- `transforms.jl` は Gaugefields に依存しない。
- `levels.jl` は renderer に依存しない。
- `renderers.jl` は Gaugefields のリンク更新を知らない。
- 配位生成は visualization の必須経路から外れる。
- 恒等配位で plaquette deviation が 0 になる。
- ランダムゲージ変換で scalar observable が不変である。
- 等値面 level が空にならず、NaN を含まない。
- GUI がない環境で軽量テストが通る。

参考文献

- [1] Akio Tomiya and contributors, *VisualizingLQCD.jl: Visualization of QCD vacuum*, GitHub repository. <https://github.com/akio-tomiya/VisualizingLQCD.jl> (accessed 2026-05-04).
- [2] Akio Tomiya and contributors, *VisualizingLQCD.jl Project.toml*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/Project.toml> (accessed 2026-05-04).
- [3] Akio Tomiya and contributors, *VisualizingLQCD.jl module source*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/src/VisualizingLQCD.jl> (accessed 2026-05-04).
- [4] Akio Tomiya and contributors, *VisualizingLQCD.jl visualization.jl*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/src/visualization.jl> (accessed 2026-05-04).
- [5] Akio Tomiya and contributors, *VisualizingLQCD.jl configuration-generation.jl*. https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/src/configuration_generation.jl (accessed 2026-05-04).
- [6] Akio Tomiya and contributors, *VisualizingLQCD.jl test/runtests.jl*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/test/runtests.jl> (accessed 2026-05-04).
- [7] Akio Tomiya and contributors, *VisualizingLQCD.jl src directory listing*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/tree/main/src> (accessed 2026-05-04).
- [8] Akio Tomiya and contributors, *VisualizingLQCD.jl header.jl*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/src/header.jl> (accessed 2026-05-04).
- [9] Akio Tomiya and contributors, *VisualizingLQCD.jl constants.jl*. <https://github.com/akio-tomiya/VisualizingLQCD.jl/blob/main/src/constants.jl> (accessed 2026-05-04).
- [10] Akio Tomiya and contributors, *Gaugefields.jl: Utilities of gauge fields*, GitHub repository. <https://github.com/akio-tomiya/Gaugefields.jl> (accessed 2026-05-04).
- [11] Akio Tomiya and contributors, *QCDDMeasurements.jl: Measurements for lattice QCD*, GitHub repository. <https://github.com/akio-tomiya/QCDDMeasurements.jl> (accessed 2026-05-04).
- [12] DESY HPC and Data for Lattice QCD, *Specification of ILDG Standards*. <https://hpc.desy.de/ildg/specifications/> (accessed 2026-05-04).
- [13] M. Lüscher, “Properties and uses of the Wilson flow in lattice QCD,” *Journal of High Energy Physics* **2010**, 071 (2010). doi:10.1007/JHEP08(2010)071; arXiv:1006.4518.
- [14] K. G. Wilson, “Confinement of quarks,” *Physical Review D* **10**, 2445–2459 (1974).

[doi:10.1103/PhysRevD.10.2445](https://doi.org/10.1103/PhysRevD.10.2445).

- [15] Y. Nagai and A. Tomiya, “JuliaQCD: Portable lattice QCD package in Julia language,” arXiv:2409.03030 [hep-lat] (2024).